Anchored Integrity and Automated Containment for Clinical Record Stores

***Abstract*—We give a clinical record store an automated containment response whose evidence a write-path attacker cannot forge. The mechanism publishes the Merkle root of record digests to a time-stamping authority outside the hospital trust domain and gates containment on cryptographic mismatch against a published root; it works only if publication verifies the anchored prefix first. We implement it on a store assembled from the MIMIC-III and MIMIC-IV demonstrations and measure it end to end. With the sweep period set equal to a 0.2 s anchoring interval, a modification injected under a running workload is detected in a median 95 ms, against an expected 100 ms (40/40 trials), and in 242 ms including the anchor round trip. A record verifies against 10^6 leaves with a 20-hash proof in 11 μs, with zero false alarms over 16,800 sampled records. Containment on a digest comparison cannot fail on unmodified data; gating it on a learned score instead contained 6 innocent sessions out of 1,203 at $\tau = 0.5$, at the cost of 26% recall. A separate process holding its own copy of the root detected every modification in 40/40 trials, including when the in-domain root was rewritten. Driven through the same client-server interface as immudb, our write path is faster at the median (0.40 against 6.41 ms) but has a heavier tail; we claim no storage-engine advantage.**

***Index Terms*—clinical data integrity, tamper-evident logging, external timestamping, automated containment, healthcare security**

I. Introduction

Healthcare records are an attractive target for outside attackers, and healthcare organisations adopt zero-trust architecture slowly because legacy clinical systems resist microsegmentation [1]. The research response has been detection-heavy: a systematic review of automated incident response in healthcare finds detection dominant, response underexplored and real-time mitigation the least developed part of the literature [17]. Detection without containment leaves the compromised artifact in place.

We address one narrow part of that gap: a clinical record store in which an outside attacker has obtained write access to the storage path. We ask what it takes for an automated response to contain a modification *and* for that containment to rest on evidence the attacker cannot forge, and we implement and measure a Merkle log over record digests whose root is published to a time-stamping authority (TSA) outside the hospital trust domain, containment being triggered by cryptographic mismatch. Our contributions are four.

- **The anchoring order is a correctness requirement.** A publication step that re-anchors the current store state without first verifying that the previously anchored prefix still hashes to its published root silently promotes a tampered state to a trusted root. This is the append-only consistency requirement of Certificate Transparency [5] applied to the publication step of a clinical store, not a new check; our contribution is to show the failure when it is omitted, refuse it over 300 tamper positions, measure the cost, and state the hole it leaves (Section VI-C).
- **Detection delay is validated and decomposed, not only derived.** We inject modifications under a running workload with real RFC 3161 anchoring and report modification-to-detection time at three intervals, including and excluding the anchor round trip, against the arithmetic expectation (Section VI-B).
- **The loop closes only with an out-of-domain verifier, and we prototype one.** An in-domain attacker controlling the code can suppress its own containment, so the loop closes only when verification is performed by a party holding an independently obtained root. We implement that separation, quantify its cost, and show it survives an in-domain root rewrite (Section VI-E).
- **The price of score gating is measured on one population, and the design is compared against an off-the-shelf store through the same interface shape.** We score a learned classifier and the deterministic rule on the same simulated sessions, and compare the two stores as client-server systems with a durable commit per write, reporting immudb's verified and unverified write paths separately because they differ by only 1% (Sections VI-D and VI-F).

What this paper does not claim. External anchoring is not a new primitive: timestamping was introduced by Haber and Stornetta [3], tamper-evident logs by Crosby and Wallach [4], and Certificate Transparency [5] and RFC 3161 [7] standardise the components we use. Our Merkle construction *is* RFC 6962's: the $0x00$/$0x01$ domain separation and the leaf-domain rule are taken from it, and RFC 9162 [6] supersedes it with the same construction. Verify-before-publish is likewise the consistency requirement of that literature. Our contribution is the measured failure and cost of omitting it, the measured delay, the verifier separation and the population-correct gating comparison, not the primitive. We claim no zero-trust architecture in the NIST SP 800-207 sense [8] our access layer lacks multi-factor authentication, device posture and continuous risk evaluation and no detection of exfiltration, ransomware or credential misuse, none of which a deterministic integrity check sees (Section III-C).

II. Related Work

A. Adoption, Logging Primitives and Clinical Detection

Tyler and Viana analyse the barriers to zero-trust transition in healthcare [1] and Sahu proposes a practitioner-led reference

model for continuous monitoring and incident response [2]; neither measures a containment path. Haber and Stornetta showed a document can be timestamped so its existence at a point in time is verifiable [3], and Crosby and Wallach gave efficient tamper-evident logs with membership proofs [4]. Certificate Transparency operationalised an append-only Merkle log at internet scale and defined the consistency proofs by which a monitor checks that a new root extends an earlier one [5], [6]; RFC 3161 defines the timestamp protocol we use [7]. The property that matters is that a log whose root is held outside the writer's trust domain cannot be silently rewritten by that writer. We use that property rather than propose it, benchmarking against an off-the-shelf implementation (Section VI-F). Xu *et al.* show anomaly-based detection of inappropriate EHR access can be evaded by an adversary who knows the detector [10], which motivates gating containment on evidence rather than a score. Blockchain-based auditing covers EHR access with purpose-based control [11], [12] and hash chaining [13], reporting neither the delay nor the cost of containment. Learned IoMT detectors [14]–[16] operate on flow features; we use one only as a score source.

B. Automated Response

Khandre and Bhide report that automated response is the least mature part of the field [17]. Ornstil presents a role-aware insider-threat system for EHRs with a three-tier response and a policy engine (a master's thesis, cited as indicative rather than peer-reviewed) [18]. Adabara *et al.* combine a random-forest flow detector with a reinforcement-learning triage agent under a governance rule engine [19], and Ahmad and Gusain automate forensics and response infrastructure as code for IoMT [20]. These are the closest works to ours, and they differ in what they measure: each reports detection quality and a response mechanism, none reports modification-to-detection delay under a live anchor, and none prices gating containment on a learned score against a deterministic trigger.

Against this work, none of the above reports modification-to-detection delay under a live anchor or a measured response path; we claim no better detection and no stronger primitive, only that the containment path is specified, ordered correctly, and measured end to end under a stated cost.

III. Threat Model

A. Adversary Capabilities

The adversary is an outside attacker with no legitimate credentials that has obtained write access to the record storage path. Three capabilities matter. **Capability A (byte modification):** it alters stored record bytes without recomputing integrity metadata, so the change is content-local. **Capability B (write-path control):** it alters bytes *and* recomputes the hashes the storage layer maintains, which live in the same trust domain; against a purely local integrity structure this is undetectable by construction, so we present no empirical finding for it the anchor's contribution is that the adversary must then alter state outside its domain, and Section VI-C covers the ordering error that would otherwise let it do so indirectly. **Capability C (code modification):** it also modifies the code that hashes, verifies and contains. A modified verifier reports success, no local mechanism detects that, and containment executed by that code can be disabled, so the loop closes only when a process the attacker does not control verifies against an independently obtained root (Section VI-E); the residual assumption is in Section VII.

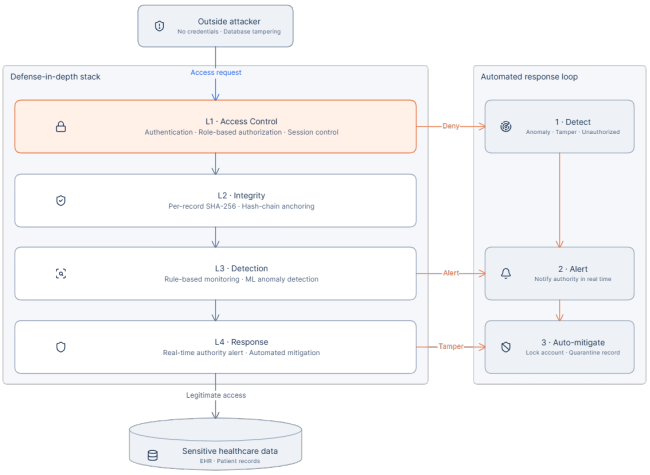


Fig. 1. Four-layer design. A request passes through L1–L4 in sequence, so the response layer acts in the request path. The Merkle root is published to an anchor outside the hospital trust boundary, which is what makes Capability B detectable.

B. Security Objectives

O1 A modification is detected at the next verification or anchoring opportunity and is attributable to a specific record. **O2** Detection rests on evidence the write-path adversary cannot fabricate without altering state outside its trust domain. **O3** Containment follows detection automatically, is bounded in cost, and does not disrupt reads of unaffected records. **O4** Modification-to-detection delay is measured under a live workload and compared with its arithmetic expectation.

C. Non-Goals

Exfiltration, ransomware and credential misuse dominate healthcare threat reports, and a deterministic check over stored bytes sees none of them: exfiltration leaves records unmodified, ransomware is indistinguishable from any other write, and a credentialed insider acting within policy produces no mismatch. We exclude all three, and our access layer is password authentication with role-based authorisation and lockout, not zero trust in the NIST SP 800-207 sense [8].

IV. Proposed Architecture

The system comprises four layers; an access request passes through them in order, so the response layer sits in the request path rather than in a separate workflow.

L1, access control. PBKDF2-HMAC-SHA256 with per-account salts and a 100,000-iteration work factor; role-based authorisation over administrator, clinician, nurse and unauthenticated outsider; lockout after repeated failures.

L2, integrity. Record digests $\mathrm{SHA256}(0x00 \,\|\, \mathrm{record})$ form the leaves of a Merkle log with RFC 6962 domain separation, internal nodes being $\mathrm{SHA256}(0x01 \,\|\, L \,\|\, R)$ [5]. The digest is stored with the record and the tree derived, so an inclusion proof is $O(\log n)$ hashes and traverses no per-record chain. The root is published to an external TSA; publication is a network operation that cannot occur per write, so the anchoring interval is a tunable setting that sets the detection bound (Section VI-B).

Verify-before-publish. A naive publication step serialises the store state, computes its root and publishes it; an adversary with Capability B that has modified records and recomputed local hashes, followed by any legitimate write triggering publication, thereby anchors the tampered state as the new trusted baseline. Publication must instead verify the previously anchored prefix: recompute the root over the first k leaves, k being the leaf count at the last publication, compare it with the root published then, and refuse on mismatch (Section VI-C, 300 tamper positions).

L3, detection. Two complementary checks run: the per-record check verifies an individual record's inclusion proof against the most recent published root ($O(\log n)$), catching content-local modification, and the anchor check recomputes the root over the anchored prefix, catching a whole-store rewrite including recomputed local hashes. The complementarity is by design the per-record check is insensitive to a coordinated rewrite, the anchor check to a single modified record whose proof is not recomputed. A learned classifier also runs in the loop (Section VI-D) as a second, lower-confidence trigger, never a substitute for the cryptographic one.

L4, response. The primary trigger is cryptographic mismatch: a failed inclusion proof or a refused publication raises an alert recording the record identifier and the evidence, revokes the active session, locks the writing account and quarantines the affected record until reviewed, following established healthcare incident-response practice [9]. The secondary trigger is a classifier score above a threshold, placing the session in the same containment sequence while reads of unaffected records continue. The primary trigger's false-containment rate is a property of the digests rather than of a threshold exactly what Section VI-D shows the secondary trigger lacks.

V. Implementation and Evaluation Setup

The prototype is Python; records live in memory in the evaluation harness, and the benchmark store persists them through a length-prefixed append log with an `fsync` per write, so "durable" means the write returned only after the bytes reached the file system there is no SQLite or WAL layer. The store is assembled from the de-identified MIMIC-III and MIMIC-IV demonstrations: 200 episodes (100 each), covering 275 admissions, 4,506 diagnoses, 107,727 laboratory events and 18,087 prescriptions from MIMIC-IV, plus 129 admissions, 1,761 diagnoses and 76,074 laboratory events from MIMIC-III. The median serialised record is 259 bytes (range 248–357), so a 32-byte digest is 12% overhead. MIMIC supplies record size, field structure and the shape of the clinical access workload, and supports no claim about detection quality. The learned score source is a Random Forest over NSL-KDD flows [21], dated and non-clinical, used only as a reproducible source of flow-feature distributions. All integrity and timing numbers come from one released script, as medians over repeated passes on one two-vCPU container.

TABLE I
Merkle log with RFC 6962 semantics. Proof length, generation and verification are per proof; the audit rebuilds the whole tree; false failures are counted over the sampled untampered records shown.

	$n = 10^5$	$n = 10^6$
Tree construction (ms)	114	731
Proof length (hashes)	17	20
Proof generation (μs)	3.2	5.5
Proof verification (μs)	9.2	11.1
Full-store audit (ms)	142	671
False verification failures	0/16800	0/16800

VI. Evaluation

A. Verification Correctness and Scale

Table I reports the integrity structure at 10^5 and 10^6 records. Proof and verification cost grow logarithmically: a proof is 20 hashes at 10^6 records and verifying one costs 11 μs the single figure this paper uses for verification, quoted unchanged in the abstract and in Section VI-G. Zero false alarms arose over 16,800 sampled untampered records at each size, and a full store audit (rebuild plus comparison against the anchored root) costs 0.67 s at 10^6. Every figure here comes from the same released run as the rest of the paper.

Two distinct defects are worth separating, because our earlier draft conflated them. The first is a *proof-construction mismatch*: our initial implementation promoted an unpaired node by duplicating it, and under that construction untampered records produced false verification failures nonzero at $n = 100$, 1,000, 10,000, 10^5 and 10^6 (4.00%, 0.80%, 18.08%, 1.70%, 1.70%), and zero whenever n is a power of two. The rate is not monotone in n and has no closed form, but one rule generates every case: walking up the tree, the highest level whose node count is odd ends in a single node with no partner, and exactly that node's leaves fail a contiguous run at the end of the store (e.g. $[983{,}040, 10^6 - 1]$ at $n = 10^6$). That is why $n = 10{,}000$ looks alarming: the unpaired node there covers 1,808 of only 10,000 leaves. We checked the failing set against the span at every size, and report the observed counts rather than a formula. The second is a *second-preimage ambiguity* of the CVE-2012-2459 kind: under duplication a tree with $2m - 1$ leaves and one with $2m$ leaves whose final leaf repeats its predecessor produce the same root, so a proof against one verifies against the other; under promotion they do not, and trees whose $2m$ leaves are all distinct collide under neither rule. That ambiguity is real but is not what produced the rates above, and it needs the adversary to supply the repeated leaf. A deployment should treat the mismatch as a liveness hazard and the ambiguity as a soundness hazard;

TABLE II

REAL RFC 3161 ANCHOR ROUND TRIP (60 REQUESTS PER VENDOR; ALL 120 GRANTED), AND MEASURED MODIFICATION-TO-DETECTION DELAY UNDER A RUNNING WORKLOAD. "TICK" STOPS THE CLOCK AT THE CATCHING VERIFICATION EVENT; "INCL. ANCHOR" RUNS THROUGH THE ROUND TRIP. EVERY INJECTED MODIFICATION WAS DETECTED: 40/40 AT EACH INTERVAL FOR TICK, 20/20, 20/20 AND 10/10 FOR INCL. ANCHOR. $T/2$ IS THE ARITHMETIC EXPECTATION FOR A RANDOM OFFSET.

Anchor round trip, median (p95 / p99), ms			
rfc3161.ai.moda	91.9 (130.9 / 143.0)	tsa.sectigo.com	2590.1 (3333.4 / 5669.2)
Modification to detection, ms (median / mean)			
T (s)	*tick*	*incl. anchor*	mean: tick + trip
0.2	95 / 100	242 / 229	100 + 129
1.0	493 / 493	614 / 621	493 + 128
5.0	2488 / 2244	2623 / 2387	2244 + 142

the RFC 6962 rule removes both, giving zero false failures from $n = 1$ to $n = 10^6$.

B. The External Anchor, and Measured Detection Delay

The anchor is a real RFC 3161 timestamp from a public TSA. Table II reports two independent public TSAs over 60 requests each: rfc3161.ai.moda returns in a median 91.9 ms (p95 130.9 ms) and tsa.sectigo.com in 2.59 s (p95 3.33 s), all 120 granted. Every request opened a fresh process and TCP/TLS connection and carried a fresh nonce, as the delay numbers assume. The choice matters: in a separate comparison the same requests took 55 ms over one reused connection against 118 ms fresh, so a connection-reusing client would see roughly half our round trip. We keep the conservative figure throughout.

The token is 6,007 bytes with a three-certificate DigiCert chain, and its chain validates: over 12 independently obtained tokens from `rfc3161.ai.moda`, the TSA used in the delay experiments, `openssl ts -verify` returned OK against the system trust store in 12 of 12 cases and failed in 12 of 12 against different data so validation is neither vacuous nor only about the imprint.

Detection delay is a property of the anchoring interval, not of the cost of a hash comparison. The store is swept periodically, and in these experiments the sweep period is set equal to the anchoring interval T, so a modification waits for the next sweep (one sweep costs 38 ms over 2,000 records: small here, not against a much shorter interval). The *tick* figure is that wait; the *anchor-inclusive* figure adds the round trip. Both used `rfc3161.ai.moda` with a fresh connection per request, and every injected modification was detected.

The tick medians (95, 493 and 2488 ms) sit on the arithmetic expectation $T/2$ (100, 500 and 2500 ms), and the anchor-inclusive figures exceed them by roughly one round trip, as they should. Because the tick delay follows from a once-per-interval sweep, this validates the schedule under a live workload and decomposes its cost; it does not discover a new relationship. The decomposition uses *means*, because means add and medians do not: the means close to within 1 ms ($100 + 129 = 229$; $493 + 128 = 621$; $2244 + 142 \approx 2387$). The 128–142 ms trips in that decomposition are higher than the 91.9 ms median of the 60-request series. They are means, so the tail pulls them up, and they come from separate runs, so we treat them as the same order of magnitude rather than as one quantity. With the slower vendor a 5 s interval should be budgeted nearer 5 s than 2.5 s.

C. The Re-Anchoring Race, and What It Does Not Cover

Verify-before-publish is tested over 300 tamper positions in three regions of the anchored prefix (100 in the first decile, 100 middle, 100 final), every one caught. A modification confined to the suffix written after the last anchor is correctly treated as new data and the prefix check passes a design limit rather than a bug: *the suffix since the last anchor is unprotected until the next anchor completes*, exactly the detection-delay window measured above.

Relation to consistency proofs, and cost. The check is the append-only consistency requirement of Certificate Transparency [5]: a new root must extend the previously published one. CT realises it with $O(\log n)$ consistency proofs; our prototype instead recomputes the root over the first k leaves from record bytes, which is $O(k)$ and also re-hashes the stored records. We did not implement consistency proofs. The consequence is a scaling limit we do not hide: a full audit costs 0.67 s at 10^6 records (Table I), longer than a 0.2 s interval, and the short-interval experiments ran over 2,000 records. The two results are therefore not combined, and a deployment wanting short intervals at that scale would need consistency proofs or an incremental check.

A refused publication raises an incident, but a prefix mismatch does not name the offending record. We localise it by recursive bisection rather than claim attribution we do not have: comparing recomputed node hashes level by level identifies the modified subtree in $O(\log n)$ comparisons per level. In a store of 1,024 leaves with block size 64, three simultaneous modifications were localised to exactly their blocks over five independent trials with randomly drawn positions (e.g. $\{16, 103, 832\} \rightarrow$ blocks $\{0, 1, 13\}$), 16 block comparisons per trial, none false and none missed. This is a demonstration at one store size, not a scaling result. Without localisation the trigger is reportable only as a store-level event.

D. Gating Containment on a Learned Score

The anchor check is deterministic, so its containment behaviour is a property of the digests; the classifier is not.

Provenance first, because our earlier draft was ambiguous about it. The score gate in Table III comes from a *separate* Random Forest of 300 trees trained on the simulated clinical sessions themselves, with a 70/30 train–test split, five session features (records accessed, distinct patients, hour of day, write ratio, off-hours flag) and 1,320 test sessions 1,203 honest, 117 tampered. It is *not* the NSL-KDD model: that classifier sees 41 network flow features and cannot score a five-feature clinical session at all, so we trained a second model. Sessions were labelled by construction, 80% of tampers subtle and 20% obvious, so the classes overlap rather than separating cleanly. An earlier attempt used a feature encoding the label directly (AUC 1.00, zero needless containments); we discarded it as accidentally rigged. Population and labels are synthetic, from

TABLE III

CONTAINMENT ON ONE POPULATION: 1,320 SYNTHETIC CLINICAL SESSIONS (1,203 HONEST, 117 TAMPERED), LABELLED BY CONSTRUCTION WITH 80% SUBTLE TAMPERS. EVERY FPR IN THIS PAPER IS OVER HONEST SESSIONS, EVERY RECALL OVER TAMPERED ONES. THE DETERMINISTIC ROW IS BY CONSTRUCTION, NOT A MEASUREMENT.

Trigger	Contained (of 1,203 honest)	FPR	Recall
Learned score, $\tau = 0.3$	45	0.0374	0.333
Learned score, $\tau = 0.5$	6	0.0050	0.256
Learned score, $\tau = 0.7$	2	0.0017	0.197
Digest mismatch	0	0.0000	1.000

MIMIC-IV field structure and admission distributions, and the caption says so.

On this population the learned score reaches AUC 0.765 and recall at the loosest threshold is 33%, against 100% for the digest check and with 45 innocent sessions contained rather than none. Tightening the threshold buys false-positive reduction at a steep cost in recall: 0.5 removes 39 of the 45 needless containments but loses a quarter of the detections it had. The design implication is that the cryptographic check is the trigger and the score a second, lower-confidence signal and it is why we rest no claim on the score's discrimination: the tampered sessions are ours, so recall partly reflects how we built them.

Two further scenarios drive the *NSL-KDD* model over *network flows*, the only score source with ground-truth labels per item, and are not comparable to Table III. That model reaches accuracy 0.783, precision 0.968, recall 0.640 and AUC 0.963 over 22,544 test flows (12,833 attacks). In the first, 4,000 flows ran through the loop with a live per-flow call (3.62 ms per flow); the gate at $\tau = 0.5$ contained 1,535 1,490 of 2,284 attacks (65%) and 45 of 1,716 honest flows (2.6%). In the second, 3,000 scored flows (1,676 attacks, 1,324 honest) plus 2 directly modified records gave 1,151 containments: 1,112 attacks (66%), 37 honest (2.8%), and the 2 records caught only by the cryptographic trigger, which is the point of pairing the two. Without the ground-truth split such a count is alert volume with no meaning; with it, it carries a false-positive rate a deterministic trigger never produces.

E. Out-of-Domain Verification

Capability C bounds the claim: an in-domain verifier can be modified, so the loop closes only if verification happens where the attacker is not. We implemented that separation as a second process that never reads the store's root, holding instead a root it obtained independently.

Three conditions of 40 trials each settled the verifier: it agreed with the in-domain verifier on unmodified data (40/40), detected a modified record (40/40), and still detected the modification when the in-domain root had been rewritten to match the tampered store (40/40) the case Capability C turns on. Its cost is small: fetching the anchored root out of band (0.23 ms), recomputing proofs from bytes it reads directly (0.22 ms) and validating against its own copy total a median 0.45 ms (p95 0.53 ms) over 5,299 bytes. We are explicit that a separate process on the same host is not a genuinely separate trust domain: it shows the detection logic does not depend on store-side state, not that an attacker with host-level access cannot reach the verifier too. That fetch is a local read, not a token chain validation. The token covers less than it may appear it binds the published root to a point in time, proving the root existed at issuance, not that it is a correct digest of good data, which follows from the prefix verification before publication.

TABLE IV

BOTH SYSTEMS AS A CLIENT-SERVER STORE WITH A DURABLE COMMIT PER WRITE AND A VERIFIED READ PER RECORD, 5 PASSES × 200 REAL RECORDS, MEDIANS AND P95. INTERFACE SHAPE MATCHED; WORK PER WRITE IS NOT. IMMUDB'S UNVERIFIED WRITE IS REPORTED SO THE GAP CANNOT BE READ AS THE COST OF ITS CONSISTENCY PROOF.

	Write		Verified read	
System	median	p95	median	p95
immudb v1.11, verified	6.41 ms	7.77 ms	0.52 ms	0.80 ms
immudb v1.11, unverified	6.33 ms	7.56 ms	0.38 ms	0.59 ms
This work	0.40 ms	5.54 ms	0.22 ms	7.03 ms

F. Comparison with an Off-the-Shelf Store

Our earlier draft compared an in-process per-record loop against immudb's client-server write, which is not a comparison. Table IV drives both systems through the same interface shape: a client issuing length-prefixed requests over loopback, one durable commit per write, one verified read per record, five passes of the same 200 real records, our client verifying all 1,000 inclusion proofs. "Durable" here means a `write+fsync` append to a log on `ext4` (`/dev/vda`, `data=ordered` a local block device, not a network mount, which is what makes a sub-millisecond median plausible). The `fsync` itself costs a median 0.12 ms (p95 0.18 ms), and the same append without it 0.001 ms, so durability is most of the write cost. We did not tune immudb's durability settings, so its figures reflect its configuration as run and are not guaranteed to match our `fsync` semantics. Because immudb verifies consistency against its previous state on every write while our store defers verification, we also drove its unverified write, so the table separates the cost of verification from the cost of the store.

The medians favour our write path, but the reason is not immudb's consistency proof: its unverified write is 6.33 ms against 6.41 ms verified, a 1% difference, so the proof does not account for the gap. The plausible explanation is that the difference lies in the store path itself, a client-server gRPC round trip with a durable commit; we did not isolate it, so we offer it as a hypothesis rather than a finding, and we claim no storage-engine advantage. The medians also hide a heavier tail on our side. Our write p95 is 5.54 ms against a 0.40 ms median, although the `fsync` costs only 0.12 ms (p95 0.18 ms), so the tail arises elsewhere in a container shared by client and server, and we did not isolate it. Our read p95 is 7.03 ms against immudb's 0.80 ms, since we build a Merkle path per read. The comparison is therefore of medians only, and the tails favour immudb. It establishes that the design does not sit on a pathologically slow store, and that its anchoring and proof semantics are not free relative to a system keeping

integrity state internal. A byte flipped in immudb's value log caused its verified read to fail with "data is corrupted: value length or digest mismatch": it detects modification in its own domain but holds no state outside it, so a coordinated rewrite of that state is not something it can detect the gap our external anchor addresses, not a speed claim. An authenticated clinical write costs about 25 ms end to end in the run that measured it, essentially all of it PBKDF2, so the storage write at 0.40 ms is under 2% of that request.

G. Cost of the Clinical Path

At $n = 10^6$, digesting the 259-byte median record costs 0.7 μs, verifying its 20-hash proof 11 μs (Table I), and the whole detect–alert–contain sequence 11.4 μs, which *includes* the verification (containment adds 0.2 μs). The durable storage write is 0.40 ms. PBKDF2 at 100,000 iterations cost 21 ms in this run and about 25 ms in the run behind the end-to-end write figure, the run-to-run variation noted in Section VII; per authenticated request, integrity work sits about 1,900$\times$ below it (21 ms basis). The dominant cost of the design is temporal rather than cryptographic: the anchoring interval that sets the detection bound.

VII. Discussion and Limitations

The guarantee is narrow: a modification that changes stored bytes is detected, against a root the writer cannot rewrite. Five bounds apply. **Not anomaly detection:** exfiltration, ransomware and credential misuse produce no digest mismatch (Section III-C). **Verifier domain:** the loop closes only with a verifier outside the attacker's domain; a same-host process is not one. **Anchoring costs liveness:** the vendor spread (91.9 ms against 2.59 s) moves end-to-end delay materially, an outage stops publication but not verification of anchored data, and the suffix since the last anchor is unprotected. **Scale:** the prefix check is $O(k)$ and was not combined with the 10^6-record experiments (Section VI-C). **Simulation:** the gating population is synthetic with a fixed 80/20 subtle-to-obvious tamper mix, so score recall partly reflects its construction, and NSL-KDD is over two decades old. Timings come from one two-vCPU container with loopback traffic and varied up to twofold across our runs, hence medians and spreads; we expect the structural relationships, not absolute values, to transfer.

VIII. Conclusion and Artifact

We gave a clinical record store a containment response resting on evidence a write-path attacker cannot forge, and measured what it costs. Two results were less comfortable than the rest: anchoring must verify the previously anchored prefix before publishing, or it promotes a tampered store to a trusted root, and gating containment on a learned score contains innocent sessions where a digest comparison contains none, which is why the cryptographic check is the trigger and the score secondary. Open work includes a genuinely independent verifier, consistency proofs in place of prefix recomputation, and evaluation on live clinical access logs in place of the synthetic population and the two-decade-old flow dataset used here. The prototype, the script producing every number here, the result files behind each table and a README mapping each table to its generating script are at https://waliulrayhan.github.io/anchored-integrity/. The MIMIC-III/IV *demonstration* subsets used here are real de-identified patient records, openly redistributable without credentialing (only the full databases need it); NSL-KDD and MIMIC are third-party and not redistributed.

References

[1] D. Tyler and T. Viana, "Trust No One? A Framework for Assisting Healthcare Organisations in Transitioning to a Zero-Trust Network Architecture," *Applied Sciences*, vol. 11, no. 16, art. 7499, 2021, doi: 10.3390/app11167499.

[2] S. K. Sahu, "Healthcare Zero-Trust Resilience Architecture (HZTRA): A Practitioner-Led Reference Model for Securing Hospital Information Systems," *Computer Fraud & Security*, pp. 1312–1316, 2025, doi: 10.52710/cfs.1095.

[3] S. Haber and W. S. Stornetta, "How to Time-Stamp a Digital Document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.

[4] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging," in *Proc. 18th USENIX Security Symposium*, Montreal, QC, Canada, Aug. 2009, pp. 317–334.

[5] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, IETF, June 2013, doi: 10.17487/RFC6962.

[6] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency Version 2.0," RFC 9162, IETF, Dec. 2021, doi: 10.17487/RFC9162.

[7] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, "Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)," RFC 3161, IETF, Aug. 2001, doi: 10.17487/RFC3161.

[8] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[9] "Healthcare Industry Cybersecurity Practices (HICP): Technical Volume 1," U.S. Department of Health and Human Services, 405(d) Program, 2023.

[10] X. Xu *et al.*, "How to Cover up Anomalous Accesses to Electronic Health Records," in *Proc. 32nd USENIX Security Symposium*, Anaheim, CA, USA, Aug. 2023.

[11] F. Ullah *et al.*, "Blockchain-Enabled EHR Access Auditing: Enhancing Healthcare Data Security," *Heliyon*, vol. 10, no. 16, art. e34407, 2024, doi: 10.1016/j.heliyon.2024.e34407.

[12] F. Ullah *et al.*, "EHR Management Evolution Through Purpose-Based Access Control and Blockchain," *Int. J. Information Security*, vol. 24, no. 1, art. 63, 2025, doi: 10.1007/s10207-024-00967-0.

[13] R. Pandey, A. K. Sahoo, and S. K. Singh, "Lightweight and Scalable Blockchain-Based Integrity Verification for EHR Systems," *Natural Resources for Human Health*, vol. 6, no. 4s, pp. 748–761, 2026, doi: 10.53365/nrfhh.824.

[14] P. Waghmode *et al.*, "Intrusion Detection System Based on Machine Learning Using Least Squares SVM," *Scientific Reports*, vol. 15, no. 1, art. 12066, 2025, doi: 10.1038/s41598-025-95621-7.

[15] H. Benahmed *et al.*, "HBiLD-IDS: An Efficient Hybrid BiLSTM-DNN Model for Real-Time Intrusion Detection," *Information*, vol. 16, no. 8, art. 669, 2025, doi: 10.3390/info16080669.

[16] N. Faruqui *et al.*, "SafetyMed: A Novel IoMT Intrusion Detection System Using CNN-LSTM Hybridization," *Electronics*, vol. 12, no. 17, art. 3541, 2023, doi: 10.3390/electronics12173541.

[17] G. Khandre and S. Bhide, "AI-Driven Automated Incident Response in Healthcare Cybersecurity: A Systematic Review of SOAR Frameworks, IoMT Security, and Emerging Trends," *Int. J. Innovative Science and Research Technology*, vol. 11, no. 5, art. IJISRT26MAY1548, 2026, doi: 10.38124/ijisrt/26may1548.

[18] C. Ornstil, "A Role-Aware Insider Threat Detection and Response System for Electronic Health Records," M.S. thesis, California Polytechnic State University, 2025.

[19] I. Adabara *et al.*, "Trustworthy and Ethical Intrusion Detection for Healthcare Internet of Medical Things Using Reinforcement Learning and Governance Rules," *Sci. Rep.*, vol. 16, no. 1, art. 25847, 2026, doi: 10.1038/s41598-026-55707-2.

[20] A. Ahmad and A. Gusain, "A Zero Trust Framework with Built-in Forensics and Response for IoMT," *LGU Res. J. Comput. Sci. IT*, vol. 10, no. 1, 2026, doi: 10.54692/lgurjcsit.2025.10176.

[21] M. Tavallaee *et al.*, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proc. IEEE CISDA*, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.